MVSS+: A Provably Secure Cross-Shard Account Migration Protocol Thwarting Replay Attacks

***Abstract*—The cross-shard account migration mechanism is one of the most advanced solutions to the unbalanced load problem in blockchain sharding systems. Existing solutions employ locking schemes to ensure the atomicity of transactions and the consistency of account states. However, these methods suffer from high transaction locking rates, degraded system performance, and disordered transaction execution sequences. To overcome these limitations, this paper proposes the first cross-shard account migration mechanism based on Multi-Version State Synchronization (MVSS), which ensures sequential execution of transactions while reducing the locking rate and latency, thereby enhancing system throughput. Specifically, MVSS divides related transactions into two categories based on their arrival time, and innovates to have the source shard and the destination shard process each category of transactions, instead of blocking all transactions at the source shard as in the existing schemes. Furthermore, in the special timestamp interleaving scenario, MVSS synchronizes the multiple version states of the account among the related shards according to the fine-grained order, which effectively maintains the consistency of the transaction sequence and the account state. Additionally, this paper delves into the security aspects of MVSS, proposing effective solutions to combat double-spending and replay attacks. Finally, We conducted extensive experiments based on a blockchain sharding simulator using over 2000 lines of *Go* to evaluate the performance of MVSS. The experimental results indicate that, compared to the state-of-the-art solutions such as Fine-tuned scheme, MVSS reduces locking time by 23.4%, decreases transaction latency by 7.5%, and improves throughput by 2.3%. This journal version introduces a delta state synchronization mechanism to optimize transmission overhead. MVSS-Delta reduces synchronization cost by 64% compared to the conference version.**

***Index Terms*—Blockchain, sharding, account migration.**

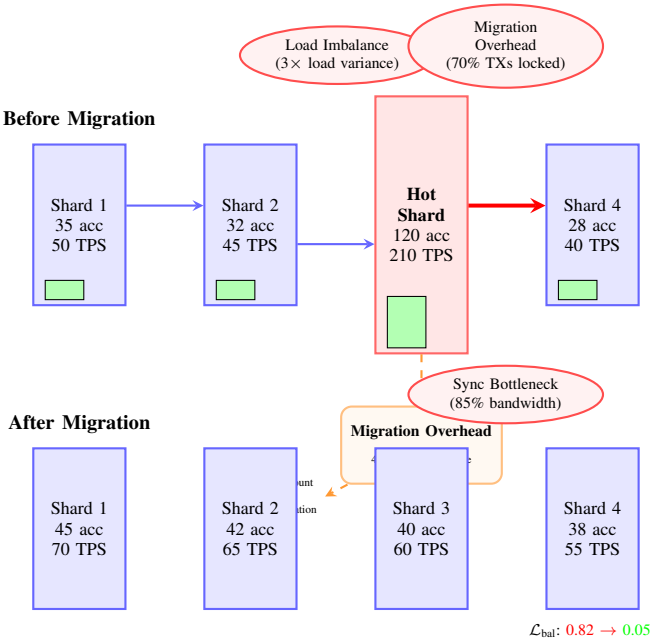


Fig. 1: Blockchain sharding challenges: (1) **Load Imbalance** - Hot shards process 3× more transactions than cold shards; (2) **Migration Overhead** - Account migration causes 40-70% latency spikes due to transaction locking; (3) **Synchronization Bottleneck** - Full-state synchronization consumes 85% of cross-shard bandwidth. The migration process aims to reduce load imbalance metric $\mathcal{L}_{\text{bal}}$ from 0.82 to 0.05.

I. Introduction

A. Background and Motivation

The decentralized nature of blockchain systems, while ensuring trustlessness and censorship resistance, inherently confronts the fundamental *scalability trilemma*—balancing decentralization, security, and performance. As application demand grows, this trilemma becomes a critical bottleneck. Sharding technology [1]–[8] has emerged as a promising solution, partitioning the network into smaller, parallel-processing *shards* to enable horizontal scaling. State sharding architectures, where each shard maintains only a subset of the global state, face fundamental limitations in dynamic load balancing. Due to the unpredictable and skewed nature of transaction workloads, certain shards become *hot shards*, processing a disproportionately high volume of transactions, while others remain underutilized. As illustrated in Fig. 1, this imbalance can be severe, with hot shards processing up to 3× the load of cold shards, creating performance hotspots and undermining the scalability benefits of sharding. Solutions such as Brokerchain [9] employ sophisticated algorithms like Metis to optimize account distribution, thereby minimizing cross-shard transactions. Building upon these foundations, cross-shard account migration schemes [10]–[12] enable dynamic account reallocation without interrupting regular transaction processing. However, existing migration mechanisms suffer from critical limitations that undermine their practical effectiveness. As summarized in Table I, current approaches typically lock 50-70% of transactions related to migrating accounts to ensure state consistency, which creates a performance bottleneck.

This paper introduces MVSS+, a novel protocol based on multi-version state synchronization that coordinates sequential transaction processing between source and target shards. Our approach significantly reduces both the proportion and duration of locked transactions while maintaining strict ordering guarantees. Furthermore, we introduce a delta state synchronization mechanism that reduces synchronization costs by 64% compared to full-state synchronization, effectively eliminating the bandwidth bottleneck in large-scale migrations.

TABLE I: PERFORMANCE OF CROSS-SHARD ACCOUNT MIGRATION PROTOCOL

Protocol	Consensus round	$\mathbf{TX_{payee}}$ Locking	$\mathbf{TX_{payer}}$ Locking	TXs disorder
Full-locked	2	Yes	Yes	Yes
Semi-locked	4	No	Yes	Yes
MVSS	**4**	**No**	**No**	**No**

B. Limitations of Prior Art

A systematic analysis of existing cross-shard account migration protocols reveals fundamental limitations across multiple critical dimensions, rooted in shared design paradigms that create inherent trade-offs.

1) Performance Overhead: The Locking Dilemma: The most severe limitation across existing approaches is their reliance on transaction locking, which creates systemic performance bottlenecks. The locking mechanism inherent in both full-locked [11] and semi-locked [12] approaches creates severe performance bottlenecks. Full-locked protocols increase transaction latency by 40-70%, while semi-locked alternatives still incur 20-50% latency overhead. This degradation stems from transactions being queued during migration completion, affecting not only migrating accounts but also causing cascading delays throughout the system. Throughput reductions of 25-35% are commonly observed due to constrained parallel processing capability. More critically, the deterministic nature of this performance penalty creates a perverse incentive to avoid migrations, ultimately undermining dynamic load balancing.

2) Consistency Guarantees: Inadequate Ordering Semantics: Current protocols provide insufficient consistency guarantees, particularly regarding transaction ordering across migration boundaries. Both categories fail to ensure deterministic transaction ordering, creating windows where logical sequences may be reordered and leading to inconsistent final states. Semi-locked approaches introduce additional complexity through differential treatment of transaction types (TX_{payer} vs. TX_{payee}), creating implicit dependencies that violate transaction semantics. Furthermore, while providing basic migration atomicity, existing protocols cannot guarantee that dependent transaction sequences execute atomically across the migration boundary, resulting in subtle consistency violations.

3) Scalability Limitations: Communication and Synchronization Bottlenecks: Architectural choices in existing protocols create fundamental scalability constraints. Full-state synchronization consumes up to 85% of cross-shard bandwidth during migration, establishing a hard scalability limit as shard counts increase. Semi-locked protocols exacerbate this issue by requiring additional consensus rounds (4 vs. 2 in full-locked), increasing migration duration and failure susceptibility. The need for complex failure recovery mechanisms and timeout handling further increases implementation complexity and reduces system robustness at scale.

4) Design Philosophy Limitations: Inherent Trade-off Assumptions: The root limitation lies in the underlying design philosophy that assumes inherent trade-offs between system properties. Both protocol types operate on the premise that maintaining consistency requires sacrificing availability through transaction locking—a zero-sum mindset that prevents exploration of alternative coordination mechanisms. Additionally, existing schemes enforce rigid temporal partitioning between pre-migration and post-migration states, treating migration as a disruptive "mode switch" rather than a continuous process. This binary approach, combined with monolithic synchronization strategies, creates predictable performance cliffs and prevents graceful handover.

These limitations motivate MVSS+, which introduces multi-version state synchronization to eliminate locking while preserving strong consistency, transforming migration from a blocking operation into a cooperative handover.

C. MVSS in a Nutshell

To address the core limitations of existing migration protocols, this paper proposes **MVSS+**, a cross-shard account migration protocol based on Multi-Version State Synchronization. MVSS+ replaces the traditional *locking-and-transfer* approach with a novel *cooperative processing* paradigm, enabling simultaneous transaction processing during migration while maintaining strong consistency. As illustrated in Figure 2, MVSS+ eliminates transaction locking through workload partitioning: the source shard processes pre-migration transactions (TX_{old}) while the target shard handles new arrivals (TX_{new}). This cooperative scheduling directly resolves the performance overhead inherent in locked protocols. To ensure consistency across potential timestamp interleaving, MVSS+ introduces *multi-version state synchronization*. When conflicts between TX_{old} and TX_{new} are detected, shards exchange cryptographically verified updates (TX_{sync}) to maintain transaction ordering and state consistency, effectively addressing the weak ordering guarantees of existing approaches. For scalability, the protocol incorporates *delta state synchronization* (MVSS-Delta), transmitting only incremental changes (Δ) rather than full states. This optimization significantly reduces the bandwidth bottlenecks that plague current solutions. Security mechanisms are integrated throughout the design to prevent migration-specific attacks, while the fundamental philosophy shift—from atomic blocking to continuous handover—overcomes the inherent trade-off assumptions in prior art. In essence, MVSS+ transforms migration from a disruptive operation into a seamless online process, simultaneously achieving the performance, consistency, and scalability goals that have eluded existing solutions.

D. Challenges and Solutions

The implementation of MVSS+ presents several fundamental challenges that must be addressed to achieve its performance and consistency objectives.

Fig. 2: High-level overview of the MVSS+ protocol, illustrating the cooperative transaction processing between source and target shards across the four stages: Preparation, Migration, Synchronization, and Broadcast.

The first challenge is how to efficiently schedule transactions cooperating to process transactions between the source and target shards, thereby reducing transaction locking. Simply partitioning workload (TX_{old} and TX_{new}) is insufficient, as timestamp interleaving can lead to state conflicts, and naively synchronizing full states would impose prohibitive communication overhead. Our solution is a unified coordination mechanism that integrates *multi-version state synchronization* with an *incremental delta-sync strategy*. When potential conflicts are detected, the shards engage in a synchronization phase. However, instead of exchanging full account states—a process that consumes up to 85% of cross-shard bandwidth in existing protocols—our MVSS-Delta optimization transmits only the cryptographically secured, incremental state changes (Δ). This design is fundamental to the protocol's viability, making frequent, fine-grained synchronization practical and thereby enabling truly non-blocking processing. To prevent disorder, the source shard, possessing the complete history of TX_{old}, acts as a decentralized ordering authority, propagating a deterministic transaction sequence to the target shard. This integrated approach ensures that the performance benefits of cooperative processing are not undone by the overhead of maintaining consistency.

The second challenge is to provide deterministic transaction ordering between source and target shards in a dynamic operating system while enabling efficient state verification. In MVSS, the independent processing speeds of different shards create inherent risks of transaction disorder, necessitating a reliable ordering mechanism. Traditional global ordering approaches [13] relying on external coordinators introduce unacceptable latency and poor real-time performance for our migration scenario, especially given MVSS's need to dynamically adjust ordering based on arriving TX_{new} transactions. Our solution establishes the source shard as the natural ordering authority, leveraging its complete knowledge of TX_{old} transactions. The source shard continuously collects timestamp information, maintains a real-time order list, and propagates deterministic sequences to the target shard. Crucially, we enhance this ordering mechanism with a *hierarchical Merkle proof* verification scheme that efficiently validates both the ordering commands and the accompanying state updates. This integrated approach ensures not only transaction consistency but also maintains the performance benefits of our cooperative processing model by minimizing the verification overhead typically associated with cross-shard coordination. The target shard can thus reliably verify the integrity and ordering of synchronized states without compromising system throughput or migration efficiency. This solution effectively addresses the dual requirements of deterministic ordering and efficient verification, enabling MVSS+ to maintain strong consistency guarantees while preserving the performance advantages of our lock-free migration approach.

E. Main Contributions

The contributions of this paper are summarized as follows:

- We propose MVSS, the first cross-shard account migration solution based on multi-version state synchronization, which reduces transaction locking while ensuring sequential execution.
- MVSS innovatively schedules related shards to process transactions cooperatively, alleviating the transaction locking issue. Additionally, we provide fine-grained order and multi-version state synchronization between relevant shards, addressing the transaction disorder problem. This paper also explores the security of MVSS, presenting solutions for various attacks.
- We implemented MVSS based on a blockchain sharding simulator and assessed its performance. Experimental results indicate that MVSS reduces locking time by 23.4%, increases throughput by 2.3%, decreases transaction latency by 7.5%, and ensures sequential execution of transactions.

The rest of this paper is organized as follows: Section II introduces the preliminaries, Section III details the MVSS mechanism, Section IV analyzes the property, Section V discusses the experimental setup and results, Section VI reviews related work, and Section VII concludes the paper.

II. PRELIMINARIES

MVSS+ operates within a state-sharded blockchain, which employs PoW for Sybil resistance [14], cuckoo rule for join-leave attack protection [3], and PBFT consensus within shards.

A. Sharding Fundamentals

Blockchain sharding [9] partitions a network of N nodes $\mathcal{V} = \{v_1, \ldots, v_N\}$ into K disjoint shards $\mathcal{S} = \{S_1, \ldots, S_K\}$ where $\cup_{i=1}^{K} S_i = \mathcal{V}$ and $S_i \cap S_j = \emptyset$ for $i \neq j$. The system operates a structured node lifecycle encompassing

three phases: node joining via Proof-of-Work puzzle solving ($H(nonce||ID) < target$) for Sybil resistance, shard assignment through cuckoo rule ($S_i = H(ID) \mod K$), and intra-shard transaction processing secured by Practical Byzantine Fault Tolerance consensus. For cross-shard transaction handling, the system employs a Relay mechanism that processes transactions locally when all involved accounts reside within the same shard, and forwards them to the appropriate destination shards otherwise [5]. Optimal sharding achieves three critical objectives. **Load Balancing** minimizes shard size variance through:

$$\mathcal{L}_{\text{bal}} = \frac{1}{K}\sum_{i=1}^{K}\left(|S_i| - \mu\right)^2, \quad \mu = \frac{N}{K} \tag{1}$$

with $\mathcal{L}_{\text{bal}} \to 0$ indicating perfect balance. **Cross-Shard Minimization** reduces inter-shard transactions:

$$\mathcal{L}_{\text{cross}} = \frac{1}{|E|}\sum_{(u,v)\in E}\mathbb{I}[\text{shard}(u) \neq \text{shard}(v)] \tag{2}$$

where E is the transaction edge set, $\mathbb{I}$ is the indicator function. **Security Assurance** maintains Byzantine fault tolerance:

$$\mathcal{L}_{\text{sec}} = \sum_{i=1}^{K}\max\left(0, \tau - f(S_i)\right), \; f(S_i) = \frac{|\{v \in S_i : \text{honest}(v)\}|}{|S_i|} \tag{3}$$

with $\tau > \frac{2}{3}$ ensuring PBFT consensus viability.

The overall sharding optimization is formulated as:

$$\min_{\mathcal{S}}\left(\alpha\mathcal{L}_{\text{bal}} + \beta\mathcal{L}_{\text{cross}} + \gamma\mathcal{L}_{\text{sec}}\right) \tag{4}$$

where α, β, γ are weighting coefficients.

B. Consistency Model for Cross-Shard Migration

Here, we formalize cross-shard account migration within a distributed KV model where account ownership transfer must preserve strict consistency semantics. A system history H consists of sequences of read $R(addr)$ and write $W(addr, state)$ operations on account states, culminating in commit or abort events. The fundamental ordering requirement is captured through real-time precedence ($\prec_{rt}$), where operation op_1 precedes op_2 if op_1's response occurs before op_2's invocation. The migration protocol must ensure linearizability, requiring that operations appear to execute atomically at some instant between their invocation and response while respecting real-time ordering. This presents the challenge for MVSS+: maintaining linearizability when transactions process concurrently across source and target shards during ownership transfer, where traditional locking approaches sacrifice availability and optimistic methods risk consistency violations.

III. SYSTEM DESIGN

A. Overview of MVSS

The MVSS+ protocol operates within a state-sharding blockchain architecture comprising two specialized shard types: the reference shard (R-shard) and multiple working shards (W-shards). **R-shard** acts as the system's control plane, responsible for macro-level load-balancing decisions. It collects transaction load metrics from all W-shards and periodically executes account redistribution algorithms (e.g., Metis, CLPA, TxAllo). The output is a state block containing the new account distribution, which is broadcast to all W-shards to initiate migration epochs. **W-shards** form the data plane, responsible for processing regular transactions and executing account migrations mandated by the R-shard. During a migration involving an account a, the W-shard currently hosting a is termed the *source shard*, while the shard designated to receive a is the *target shard*.

B. Data Structures for Cross-Shard Migration

To orchestrate cross-shard account migration without global locking, MVSS+ introduces three auxiliary transaction types, extending prior work [12]: (i) the initial migration transaction TX_{ini} sent by the source shard, (ii) the migration acknowledgment transaction TX_{ack} sent by the target shard, and (iii) the account version synchronization transaction TX_{sync} which can be sent by both the source and target shards.

1) Initial Migration Transaction TX_{ini}: The source shard initiates migration by generating and sending $TX_{ini} = \{\text{Target}, \text{Addr}, State_{ini}, \pi_{ini}, Sync, \mathbf{LatestMR}, \mathbf{LastCN}\}$ to the target shard, where $Target$ denotes the ID of the target shard, $Addr$ is the migrating account's address, $State_{ini}$ is the state snapshot with Merkle proof π_{ini}, and $Sync$ is a Boolean flag indicating whether state synchronization is required. **LatestMR** commits all processed transactions in the source shard's latest block, while **LastCN** records the highest committed nonce of the account, establishing a definitive baseline for incremental synchronization and preventing double-spending.

2) Migration Acknowledgment Transaction TX_{ack}: confirms reception with $TX_{ack} = \{TX_{ini}, \tau_{ini}, Abort, \mathbf{TargetBH}\}$, where τ_{ini} is the Merkle proof of TX_{ini} in the target shard's blockchain, $Abort$ indicates migration status, and **TargetBH** is the hash of the block containing TX_{ack} in the target shard, providing a precise reference for cross-shard verification.

3) Account Version Synchronization Transaction TX_{sync}: enables efficient state synchronization during timestamp interleaving scenarios, $TX_{sync} = \{State_{old}, State_{new}, \pi_{old}, \pi_{new}, \mathbf{DeltaMR}, \mathbf{StartN}, \mathbf{EndN}\}$ Where $State_{old}$ and $State_{new}$ denote the migrating account's states with corresponding Merkle proofs π_{old} and π_{new}, **DeltaMR** commits all transactions inducing the transition from $State_{old}$ to $State_{new}$, enabling efficient verification without raw data exchange. This compact commitment allows efficient verification without transmitting entire transactions, reducing bandwidth overhead. **StartN** and **EndN** specify the nonce range of the synchronized transaction batch.

For high-frequency migration scenarios, MVSS+ further employs delta synchronization $TX_{\text{sync\_delta}}$ and its associated procedures, which can be used as an alternative to the full TX_{sync} in the synchronization protocol. $TX_{\text{sync\_delta}} = \{\text{Addr}, \Delta\text{Balance}, \Delta\text{Nonce}, \text{StorageRoot}, \pi_{\text{delta}}\}$, ΔBalance denotes the signed balance change, ΔNonce is the nonce

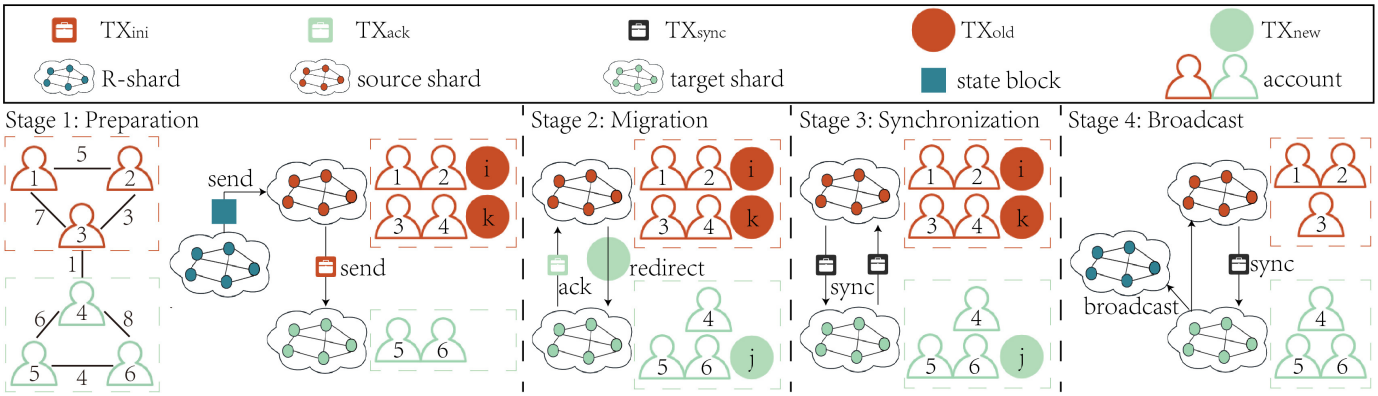


Fig. 3: An overview of MVSS.

increment, StorageRoot is transmitted only when contract storage changes, and π_{delta} is the Merkle proof for the delta states. The delta synchronization mechanism operates in sliding windows (default 100ms) to aggregate state changes, significantly reducing the number of synchronization messages required during account migration.

C. Detail of MVSS

MVSS+ achieves linearizability through an integrated protocol that coordinates three core mechanisms across four operational stages. The protocol employs **Global Transaction Ordering** maintained by the source shard $L_{order} = [tx_1, tx_2, ..., tx_n]$, based on client-generated timestamps and intra-shard consensus. This order list serves as the single source of truth for transaction sequencing across both shards. In **Multi-Version State Management**, each account state is versioned $State = \{v_1, v_2, ..., v_k\}$, where $v_i = (balance_i, nonce_i, timestamp_i, \Delta MerkleRoot_i)$. Transactions execute against specific state versions according to their position in L_{order}. As for **Atomic State Synchronization**, when timestamp interleaving is detected, TX_{sync} transactions atomically propagate state versions between shards $\text{Sync}(v_{source}, v_{target}) \rightarrow v_{merged}$, where v_{merged} insures all valid state transitions while preserving the global order.

The protocol specifically addresses **timestamp interleaving** scenarios where TX_{old} may have later timestamps than TX_{new}. Consider three transactions TX_i, TX_j, and TX_k where TX_i and TX_k belong to TX_{old} while TX_j belongs to TX_{new}, with timestamps satisfying $TS^i_{old} < TS^j_{new} < TS^k_{old}$. To ensure correct execution order TX^i_{old}-TX^j_{new}-TX^k_{old}, both source and target shards must synchronize account states after processing TX^i_{old} and TX^j_{new}, guaranteeing transactions execute against appropriate state versions.

We divide the MVSS++ protocol into 4 stages:

Stage 1: Preparation. The source shard obtains the state block provided by the R-shard, parses it, and retrieves the account partition results for the new epoch, identifying the accounts that require migration. The source shard then records the initial state of the migrating account $State_{ini}$ and generates TX_{ini}. It also computes the **LatestMR** from its most recent block and records the **LastCN** for the migrating account. If there are unprocessed TX_{old} for that account in the source shard, it generates an order list based on the timestamp information of transactions created by the client. After reaching consensus, TX_{ini} is sent to the target shard. During the migration process, the source shard merely records the transaction information, updates the order list, and then forwards with a redirection tag to the target shard.

Stage 2: Migration. Nodes in the target shard validate the effectiveness of TX_{ini} and reach consensus. If TX_{ini} is invalid, the failure handling procedure is triggered, halting the migration in the target shard, generates a TX_{ack} with $Abort$ set to true, broadcasts TX_{ack} to all shards, and redirects all transactions related to the migrated account back to the source shard. If TX_{ini} is valid, the target shard creates a temporary account and ensures that the state data of that account (balance, nonce, etc.) is consistent with the initial state provided by the source shard. The target shard also records the **TargetBH** when generating TX_{ack}. If the $Sync$ value in TX_{ini} is false, the target shard generates TX_{ack}, sets $Abort$ to false, reaches consensus within the shard, and broadcasts this to all shards, notifying that the account has been migrated to the target shard and concluding the cross-shard account migration process. If $Sync$ is true, the process proceeds to the next stage.

Stage 3: Synchronization. The source shard continuously updates L_{order}. Upon detecting interleaving, it holds back later TX_{old} (e.g., TX^k_{old}). After processing TX^i_{old}, it computes the **DeltaMR** for the state transition to $State_{ini}$, along with the **StartN** and **EndN** values, and sends TX_{sync} to the target shard. The target shard verifies the proofs (π_{old}, π_{new}) and the **DeltaMR** before updating its temporary account state. It then processes TX^j_{new}, computes a new **DeltaMR**, and sends TX_{sync} back to the source shard. This synchronization continues until all TX_{old} are processed. The final state is synchronized, and the source shard deletes the account.

Stage 4: Broadcast. After the target shard updates the final state of the account, it converts the temporary account to a normal account and broadcasts message about the successful account migration to all shards, thus completing the cross-shard account migration process. Subsequently, all related transactions are handled by the target shard.

Algorithm 1: MVSS++ Protocol

Input: Source shard S_s with account a, target shard S_d, ordered transaction list L_{order}
Output: Synchronized account $State_{final}$ in S_d

begin
  for *each migrating account* a **do**
    if *timestamp interleaving detected in* L_{order} **then**
      // S_s processes and synchronizes
      S_s: Wait for commitment of TX^i_{old};
      S_s: Compute **DeltaMR** for processed transactions;
      $S_s \rightarrow S_d$: Send TX_{sync} with $State_{old}$, $State_{new}$, **DeltaMR**, π_{old}, π_{new};
      // S_d verifies and updates state
      S_d: Verify π_{old}, π_{new}, **DeltaMR**;
      S_d: Update temporary account state;
      S_d: Execute TX^j_{new};
      S_d: Compute new **DeltaMR** for state changes;
      $S_d \rightarrow S_s$: Send TX_{sync} with updated state and proofs;
    if TX_{old} *with larger timestamp committed in* S_s **then**
      Abort conflicting TX_{new} with smaller timestamps in S_d;
    else
      S_s: Process remaining TX_{old} transactions;
      S_s: Compute final **DeltaMR**;
      $S_s \rightarrow S_d$: Send final TX_{sync} with complete state transfer;
      migration complete $\leftarrow$ **true**;
  return $State_{final}$ from S_d;

Algorithm 2: Delta State Synchronization Protocol

Input: Migrating account a with $State_{prev}$ (last synchronization) and $State_{curr}$ (current state)
Output: Delta snapshot Δ, $\emptyset$ if no changes

Function *GenerateDelta(account a)*:
  $\delta_{balance} \leftarrow State_{curr}.balance - State_{prev}.balance$;
  $\delta_{nonce} \leftarrow State_{curr}.nonce - State_{prev}.nonce$;
  $\Delta_{storage} \leftarrow \emptyset$;
  if $a.isContract() \wedge State_{curr}.storageRoot \neq State_{prev}.storageRoot$ **then**
    $\Delta_{storage} \leftarrow$ ComputeStorageDelta($State_{prev}.storageRoot$, State$_{curr}.storageRoot$);
  if $\delta_{balance} = 0 \wedge \delta_{nonce} = 0 \wedge \Delta_{storage} = \emptyset$ **then**
    return $\emptyset$
  $\pi_{delta} \leftarrow$ GenerateMerkleProof($State_{curr}$)
  return $\langle \delta_{balance}, \delta_{nonce}, \Delta_{storage}, \pi_{delta} \rangle$;
Function *Synchronization(Source/Target Shard* S_s*/*S_d*)*:
  $window \leftarrow$ InitializeSlidingWindow(100ms)
  $\Delta_{aggregate} \leftarrow \emptyset$;
  while ***not*** *migration_complete* **do**
    foreach *migrating account* a *in* S_s **do**
      $\Delta \leftarrow$ GenerateDelta(a);
      if $\Delta \neq \emptyset$ **then**
        $\Delta_{aggregate} \leftarrow \Delta_{aggregate} \cup \{\langle a, \Delta \rangle\}$;
    if WindowExpired($window$) $\vee \Delta_{aggregate} \neq \emptyset$ **then**
      $TX_{sync\_delta} \leftarrow$ ConstructTx($\Delta_{aggregate}$);
      SendToShard(S_d, TX_{sync_delta})
      ResetWindow(window);
      $\Delta_{aggregate} \leftarrow \emptyset$

IV. Delta State Synchronization Mechanism

Instead of transferring complete account states, MVSS++ transmits only incremental changes to reduce bandwidth consumption while maintaining consistency guarantees.

A. Incremental Snapshot Generation

Definition 1 (State Delta)**.** *For migrating account a epoch transition $t-1 \rightarrow t$, the state delta Δ^a_t captures incremental changes, where $\delta_{balance} = balance_t - balance_{t-1}$, $\delta_{nonce} = nonce_t - nonce_{t-1}$, $\Delta_{storage}$ represents contract storage modifications, and π_{delta} provides cryptographic verification:*

$$\Delta^a_t = (\delta_{balance}, \delta_{nonce}, \Delta_{storage}, \pi_{delta}) \tag{5}$$

The storage delta computation uses *Merkle Patricia Trie*:

$$\Delta_s = \{(path_i, value_i) \mid \text{MPT}_t[path_i] \neq \text{MPT}_{t-1}[path_i]\} \tag{6}$$

B. Adaptive Window Aggregation

Definition 2 (Aggregation Window)**.** *Let W be a time-bound window parameter. The aggregation set $\mathcal{A}_W$ for shard S is:*

$$\mathcal{A}_W = \bigcup_{t \in [\tau, \tau+W]} \{(a, \Delta^a_t) \mid a \in \mathcal{M} \wedge \Delta^a_t \neq \emptyset\} \tag{7}$$

where $\mathcal{M}$ is the set of migrating accounts.

C. Timestamp Interleaving Resolution

Theorem 1 (Ordered Delta Application)**.** *For two transactions TX_i and TX_j with $TS_i < TS_j$, their deltas must satisfy:*

$$\Delta^a_{TS_j} = f(Apply(\Delta^a_{TS_i}, state_{TS_i}) \tag{8}$$

where f is the state transition function.

The validation function ensures atomicity by verifying: the new state must maintain a non-negative balance ($S_{new}.balance \geq 0$), the nonce must increment exactly by

Algorithm 3: Adaptive Window Management

Input: Current aggregation set $\mathcal{A}$, network conditions
Output: Optimal window size $W_{optimal}$

Function *ComputeAdaptiveWindowSize()*:
  $bandwidth \leftarrow$ EstimateCrossShardBandwidth;
  $txRate \leftarrow$ GetTransactionRate($\mathcal{M}$);
  $queueSize \leftarrow$ GetPendingTransactionCount;
  if $queueSize > \theta_{high}$ **then**
    return W_{min}; // Minimal window under high load
  else if $bandwidth < \theta_{bw}$ **then**
    return $\min\left(W_{max}, \frac{\theta_{bw}}{txRate}\right)$; // Bandwidth-constrained window
  else
    return $W_{default}$

one from the previous state ($S_{new}.nonce = S_{old}.nonce + 1$), and the storage root must remain valid. Only when all these conditions are satisfied does the validation function return true.

D. Delta Chain Verification

Definition 3 (Delta Chain Integrity)**.** *A sequence of deltas* $\{\Delta_1, \ldots, \Delta_n\}$ *forms a valid chain iff:*

$$H(\Delta_i) = H\left(\delta_b^{(i)} \parallel \delta_n^{(i)} \parallel H(\Delta_s^{(i)}) \parallel H(\Delta_{i-1})\right) \quad (9)$$

with $H(\Delta_0) = Hash(state_0)$.

V. System Analysis

This section presents a comprehensive analysis of the MVSS+ protocol, examining its security foundations, consistency guarantees, and formal proofs for core properties.

A. Threat Model and Security Foundations

MVSS+ rests upon the following *System Assumptions*: 1) **Honest Majority.** Each shard S_i contains at least $\lceil 2|S_i|/3 \rceil + 1$ honest nodes, a prerequisite for the security of the underlying PBFT consensus [15], [16]; 2) **Partial Synchrony.** The network is partially synchronous [17]. After an unknown Global Stabilization Time (GST), all messages between honest nodes are delivered within a known bounded delay Δ; 3) **Secure Cryptography.** Cryptographic primitives, such as hash functions and digital signatures, are collision-resistant and existentially unforgeable under chosen attacks.

We consider a computationally bounded Byzantine adversary $\mathcal{A}$ that can: 1) Adaptively corrupt up to $f < |S_i|/3$ nodes in any shard S_i, causing them to arbitrarily deviate from the protocol; 2) Control a subset of user accounts, including initiating transactions and migration requests from them; 3) Fully control the network communication between corrupted nodes. For communication involving honest nodes, $\mathcal{A}$ can eavesdrop, delay, drop, or replay messages, but cannot indefinitely prevent communication between them. The adversary $\mathcal{A}$ cannot break the cryptographic primitives and corrupt more than one-third of the nodes in any single shard.

Algorithm 4: Delta State Application and Verification

Input: Account a, state delta Δ, timestamp TS, initial state $state_0$, delta chain Δ_{chain}
Output: Application status and verification result

Function `ApplyOrderedDelta(`a, Δ, TS`)`:
  AcquireLock(a);
  $currentState \leftarrow$ GetCurrentState(a);
  if $TS < currentState.lastAppliedTimestamp$ **then**
    RejectDelta(Δ, "Timestamp violation");
    return false; // Out-of-order
  $newState \leftarrow$ ComputeNewState($currentState, \Delta$);
  if ValidateStateTransition($currentState, newState$) **then**
    UpdateAccountState($a, newState, TS$);
    ReleaseLock(a);
    return true; // Application success
  else
    RejectDelta(Δ, "Invalid state transition");
    return false; // Validation failed

Function `VerifyDeltaChain(`$state_0$, Δ_{chain}`)`:
  $previousHash \leftarrow H(state_0)$;
  foreach $\Delta \in \Delta_{chain}$ **do**
    $computedHash \leftarrow H(\Delta.\delta_b \parallel \Delta.\delta_n \parallel H(\Delta.\Delta_{storage}) \parallel previousHash)$;
    if $computedHash \neq \Delta.headerHash$ **then**
      return false; // Chain violation
    $previousHash \leftarrow computedHash$;
  return true; // Successfully verified

The MVSS+ protocol is designed to achieve the following *Security Goals*: 1) **Consistency (Safety).** All honest nodes across all shards maintain a consistent view of the state of any migrated account. No double-spending, overspending, or invalid state transitions can occur. 2) **Liveness.** All valid transactions and migration requests initiated by honest users are eventually processed and committed to the blockchain. 3) **Ordering Fairness.** The relative order of transactions pertaining to a specific account is preserved according to a deterministic protocol-defined ordering (e.g., based on timestamps), preventing disorderly execution. 4) **Auditability.** The entire account migration process is verifiable. An immutable cryptographic audit trail is maintained on-chain, allowing any node to validate the correctness of state transitions.

B. Attack Vectors and Mitigation Strategies

Within the described threat model, we analyze two critical attack vectors and detail MVSS+'s countermeasures.

1) *Double-Spending Attack Mitigation.*

Attack Vector: An adversary controlling account α with balance B_α initiates transaction $t_1 : \alpha \xrightarrow{B_\alpha} \beta$ in the source shard S_{src} prior to migration initiation. The adversary strategically delays t_1's inclusion in the blockchain, leaving it pending in the transaction pool. Concurrently, migration of α to target shard S_{tgt} commences, resulting in state replication where α's balance B_α is transferred to S_{tgt}. The adversary then exploits this temporal window by submitting transaction $t_2 : \alpha \xrightarrow{B_\alpha} \gamma$ to S_{tgt}, attempting to spend the same funds twice.

Mitigation in MVSS+: The protocol leverages the sequential monotonicity of account nonces, where each transaction consumes exactly one nonce value. The source shard S_{src}, serving as the sequencing authority for pre-migration transactions (TX_{old}), maintains continuous monitoring of nonce progression for migrating accounts. During the redirection phase, when a post-migration transaction (TX_{new}) arrives at S_{tgt} with nonce n, the target shard performs cross-shard nonce validation against the latest committed state from S_{src}. If $n \leq n_{current}$ (where $n_{current}$ represents the highest nonce processed by S_{src}), the transaction is immediately rejected as a potential double-spend attempt. This nonce-based defense is complemented by the state synchronization mechanism (TX_{sync}), which atomically propagates nonce increments between shards, ensuring global consistency of the nonce sequence space.

2) *Replay Attack Mitigation.*

Attack Vector: The transaction replay attack exploits the state transition discontinuity inherent in account migration processes. An adversary intercepts a legitimate transaction $t_1 : \alpha \xrightarrow{v} \beta$ that was successfully executed and committed in S_{src} prior to migration completion. Following α's migration to S_{tgt}, the adversary resubmits t_1 to S_{tgt}, attempting to re-execute the same state transition and illegitimately deduct value v from α's balance a second time.

Mitigation in MVSS+: MVSS+ employs a dual-mechanism defense strategy against replay attacks, combining cryptographic transaction provenance with state-aware validation. First, the protocol introduces a cryptographically secured *redirection tag* τ that is attached to all TX_{new} transactions. This tag is generated using a key derivation function $KDF(sk_\alpha, nonce_{migration})$, where sk_α is the account's private key and $nonce_{migration}$ is a migration-specific nonce. Any transaction without this valid tag is identified as a potential replay attempt and rejected by S_{tgt}. Second, the protocol enforces strict nonce continuity across migration boundaries. During migration finalization, S_{tgt} initializes α's nonce to $n_{final} + 1$, where n_{final} is the highest nonce committed in S_{src}. Consequently, any replayed transaction with nonce $n \leq n_{final}$ fails validation due to nonce violation. This combined approach of cryptographic provenance marking and state continuity enforcement provides robust protection against replay attacks while maintaining the protocol's performance objectives of minimal latency overhead.

C. Protocol Properties Analysis

Definition 4 (Protocol Liveness)**.** *A cross-shard account migration protocol satisfies* liveness *if for every migration request m initiated by an honest node at time t, there exists a finite time $t' > t$ such that either m is successfully completed and committed across all relevant shards, or m is explicitly aborted with a verifiable proof of failure distributed to all participants.*

Theorem 2. *Under the honest majority assumption ($f < n/3$ Byzantine nodes per shard) and partial synchrony, MVSS+ guarantees liveness for all valid migration requests.*

Proof: We prove liveness through protocol stage analysis: In *Stage 1 (Preparation)*, the migration initiation transaction TX_{ini} undergoes PBFT consensus within the target shard. Given $f < n/3$, honest nodes can reach agreement on TX_{ini}'s validity within bounded rounds. The consensus termination property ensures that this stage completes. In *Stage 2 (Migration)*, cooperative processing commences with the source shard handling TX_{old} and target shard processing TX_{new}. Partial synchrony guarantees message delivery within Δ after GST, preventing indefinite blocking. The absence of transaction locking eliminates the primary source of livelock in traditional approaches. In *Stage 3 (Synchronization)*, TX_{sync} operations resolve state conflicts incrementally. The sliding window aggregation with $W \geq 3$ ensures bounded synchronization latency while maintaining safety. In *Stage 4 (Broadcast)*, final state commitment employs PBFT consensus, which terminates under our fault assumptions. The abort mechanism in Stage 2 provides explicit failure notification when migrations cannot proceed. The protocol's progress across all stages is ensured by the combination of consensus termination, bounded network delays, and the elimination of blocking operations. □

Definition 5 (State Consistency)**.** *A sharded blockchain maintains* state consistency *if for any account α and any two honest nodes n_i, n_j in possibly different shards, the observed state of α satisfies:*

$$\forall t > t_{final}, \quad state_{n_i}(\alpha, t) = state_{n_j}(\alpha, t)$$

where t_{final} is the time when all transactions involving α are committed.

Theorem 3. *MVSS+ ensures state consistency for migrated accounts under the honest majority assumption.*

Proof: Consistency is preserved through three complementary mechanisms. *Sequential Nonce Enforcement:* Each transaction consumes a unique nonce n_i, preventing double-execution. The nonce sequence $n_1, n_2, \ldots, n_k$ is strictly monotonic and globally observable. *Atomic State Synchronization:* TX_{sync} operations employ two-phase coordination between source and target shards. For state transition $\delta : S \to S'$, the operation either commits δ atomically across both shards or aborts without partial effects. *Deterministic Ordering:* The source shard maintains total order L_{order} through PBFT-backed consensus. Cross-shard coordination ensures this order is preserved during migration. Formally, for transactions tx_i, tx_j with $timestamp(tx_i) < timestamp(tx_j)$, execution order satisfies $tx_i \prec tx_j$ in all shards. The combination

of these mechanisms ensures that all honest nodes observe identical state sequences for migrated accounts. □

Definition 6 (System Execution Model)**.** *An* execution history H *is a tuple* $(E, \prec_{proc}, \prec_{rt})$ *where:*

- E *is a set of events representing operation invocations and responses*
- $\prec_{proc}$ *is the program order per process (shard)*
- $\prec_{rt}$ *is the real-time order between events*

Each transaction $tx \in H$ *is modeled as a sequence of operations on account states with explicit migration events.*

Definition 7 (Migration Serializability)**.** *A history* H *is* linearizable *if there exists a legal sequential permutation* S *of* H *that preserves both process order* ($\prec_{proc}$) *and real-time order* ($\prec_{rt}$)*, ensuring operations appear to take effect instantaneously at some point between their invocation and response.*

Theorem 4 (MVSS+ Serializability Guarantee)**.** *MVSS+ guarantees linearizability for all transactions involving migrating accounts, provided the underlying intra-shard consensus is linearizable and the network is partially synchronous.*

Proof: The protocol establishes a total order L_{order} through PBFT-backed consensus, providing deterministic sequencing across shards. This ordering serves as the single source of truth for transaction sequencing. Consider any two transactions tx_i and tx_j with $tx_i \prec_{rt} tx_j$: **Same-Shard Transactions:** Local linearizability inherited from PBFT consensus ensures $\prec_{rt}$ preservation. **Cross-Shard Migration** ($TX_{old} \prec_{rt} TX_{new}$)**:** Protocol design guarantees TX_{old} commitment before TX_{new} processing through the migration initiation barrier (TX_{ini}). **Temporal Impossibility** ($TX_{new} \prec_{rt} TX_{old}$)**:** This ordering is architecturally prevented by the migration protocol's phase separation. **Atomic State Management:** Multi-version state management with $State = \{v_1, v_2, \ldots, v_k\}$ enables concurrent processing while maintaining deterministic state evolution according to L_{order}.

The combination ensures the existence of a legal sequential permutation S that respects all real-time constraints while maintaining the protocol's performance advantages. □

Theorem 5 (Atomic Migration Semantics)**.** *The account migration operation in MVSS+ is atomic—it either completes entirely with all associated state transitions or aborts without partial effects, maintaining system-wide consistency.*

Proof: Atomicity is achieved through a multi-phase commit protocol with explicit failure handling: **Preparation Phase:** TX_{ini} validation through PBFT consensus provides all-or-nothing initiation. Failure at this stage results in a clean abort with no state changes. **Incremental Synchronization:** Each TX_{sync} operation atomically applies state deltas with rollback capability through versioned state management. The hierarchical Merkle proofs provide cryptographic guarantees of state consistency throughout this phase. **Final Commitment:** The migration concludes with an atomic broadcast that either commits the final state across all shards, making the migration permanent, or triggers the abort protocol, rolling back all intermediate states using the complete version history. The protocol maintains atomicity without resorting to transaction locking, instead leveraging consensus atomicity, cryptographic state commitments, and explicit rollback procedures to achieve the same semantic guarantees. □

Definition 8 (Cryptographic Auditability)**.** *A migration protocol provides* cryptographic auditability *if any state transition* δ *can be efficiently verified using compact proofs* π_δ *such that:*

$$\Pr[\textit{Verify}(\pi_\delta, \delta) = \textit{accept} \wedge \delta \textit{ is invalid}] \leq \epsilon(\lambda)$$

where $\epsilon(\lambda)$ *is negligible in the security parameter* λ*, and* $|\pi_\delta| \in O(\log n)$*.*

Theorem 6 (MVSS+ Auditability Guarantee)**.** *MVSS+ with hierarchical Merkle proofs achieves cryptographic auditability with* $\epsilon(\lambda)$ *negligible under collision-resistant hash.*

Proof: The auditability guarantee stems from the hierarchical Merkle tree construction. Each TX_{sync} contains $\Delta\text{MerkleRoot}_{sync}$, a cryptographic commitment to the batched state transitions. For k transactions in a synchronization batch, the root provides $O(1)$ storage overhead. Verification of transaction tx_i's inclusion requires a Merkle path of length $O(\log k)$, providing logarithmic scaling relative to batch size. Suppose an adversary $\mathcal{A}$ can produce a valid proof π for invalid state transition δ^*. This implies $\mathcal{A}$ found either a Merkle tree collision: $H(x) = H(y)$ for $x \neq y$, violating collision resistance, or a valid signature for δ^* without the private key, violating existential unforgeability. Both events occur with negligible probability under our cryptographic assumptions, thus establishing the auditability guarantee. □

Theorem 7 (Storage Optimization Bound)**.** *For non-contract accounts, MVSS+ reduces state synchronization overhead to at most 14.3% of full state transfer while maintaining equivalent security guarantees.*

Proof: We analyze the information-theoretic minimal representation required for secure state synchronization. A complete account state requires:

$$\begin{aligned} |S_{full}| &= |addr| + |balance| + |nonce| + |metadata| \\ &= 32B + 32B + 8B + 56B = 128B \end{aligned}$$

MVSS+ transmits only incremental changes:

$$\begin{aligned} |S_\Delta| &= |\Delta balance| + |\Delta nonce| + |auth| \\ &= 4B + 2B + |\Delta MerkleRoot| \\ &\leq 6B + 32B = 38B \end{aligned}$$

The optimization ratio is bounded by:

$$\frac{|S_\Delta|}{|S_{full}|} \leq \frac{38}{128} \approx 29.7\%$$

For common cases with balance-only changes, the ratio improves to:

$$\frac{4B + 2B}{42B} \approx 14.3\%$$

The security is preserved because the ΔMerkleRoot commits to the complete state transition history, enabling full reconstruction when necessary. □

Theorem 8 (Sliding Window Security)**.** *With window size $W \geq 3$, MVSS+'s sliding window aggregation ensures that the probability of undetected timestamp interleaving satisfies:*

$$P_{interleave} \leq \frac{1}{2^{W-1}} + \epsilon(\lambda)$$

where $\epsilon(\lambda)$ is negligible in the security parameter λ.

Proof: The security analysis considers two attack vectors. *Timestamp Manipulation:* An adversary attempting to hide interleaving must manipulate timestamps across at least $W-1$ consecutive synchronization points. Given the consensus-based timestamp validation, the probability of successful manipulation decreases exponentially with W. *Consensus Compromise:* For undetected interleaving to persist, the adversary must maintain consensus control over the ordering for W consecutive rounds. Under an honest majority, this probability is bounded by $\left(\frac{f}{n}\right)^{W-1}$ for adaptive adversaries. Combining these bounds and accounting for cryptographic assumptions:

$$P_{interleave} \leq \max\left(\frac{1}{2^{W-1}}, \left(\frac{f}{n}\right)^{W-1}\right) + \epsilon(\lambda)$$

With $f < n/3$ and $W \geq 3$, we obtain the stated bound of $P_{interleave} < 12.5\% + \epsilon(\lambda)$. □

Theorem 9 (Performance-Consistency Trade-off Optimality)**.** *MVSS+ achieves an optimal trade-off between consistency strength and performance overhead, providing linearizability guarantees with sub-linear synchronization costs.*

Proof: MVSS+ provides linearizability, the strongest single-object consistency guarantee, equivalent to non-sharded systems. The protocol communication complexity is $O(\log k)$ for k transactions in synchronization batches, the storage overhead is 14.3% of full state transfer for common cases, the verification cost is $O(\log n)$ for state transition proofs. The $\Omega(\log n)$ lower bound for cryptographic state verification implies MVSS+ achieves asymptotically optimal verification complexity while providing maximal consistency guarantees. The protocol thus achieves the strongest possible consistency semantics without the performance penalties of traditional locking-based approaches, demonstrating an optimal trade-off in the design space of cross-shard migration protocols. □

Corollary 1 (Strong Consistency)**.** *MVSS+ maintains strong consistency guarantees even under high concurrency and frequent migration scenarios, providing the same semantic guarantees as a single-shard system for all migrated accounts.*

Proof: The consistency mechanisms in MVSS+ are designed to be compositionally correct. The global ordering L_{order} ensures that transactions are totally ordered regardless of concurrency levels. The versioned state management and atomic synchronization preserve consistency across migration boundaries. The hierarchical Merkle proofs provide efficient verification of these consistency guarantees without compromising performance. Therefore, the protocol maintains strong consistency semantics equivalent to a non-sharded system for all account operations, including those undergoing migration. □

VI. PERFORMANCE EVALUATION

A. System Implementation

We implemented MVSS+ using over 2000 lines of Go code based on the open-source blockchain sharding simulator BlockEmulator [18]. Our implementation extends the base simulator with the complete MVSS+ protocol stack, including the delta state synchronization mechanism. The experimental platform consists of 32 physical servers, each equipped with 32-core Intel Xeon Gold 6230 processors, 128GB RAM, and connected via 10Gbps Ethernet, simulating a realistic distributed environment.

The transaction workload is generated using historical Ethereum data from XBlock [19], comprising 200,000 real transactions with diverse patterns including DeFi operations, NFT transfers, and regular payments. To ensure statistical significance, each experiment is repeated 10 times, and we report the average values with 95% confidence intervals.

We compare MVSS+ against three state-of-the-art cross-shard account migration protocols: **LB [11]:** A representative full-locked scheme that locks all transactions during migration. **Fine-tuned [12]:** A semi-locked mechanism that only locks debit transactions. **EFShard [20]:** A recent protocol supporting partial state transfer, included as an additional baseline.

The experimental configuration follows realistic blockchain sharding parameters: **Shard Configuration:** 4-16 working shards (W-shards) with 64 nodes per shard **Block Parameters:** Block size of 500 transactions, block interval of 2 seconds **Transaction Load:** Injection rate from 200 to 2000 transactions per second (TPS) **Migration Trigger:** R-shard initiates migration every 50,000 confirmed transactions using CLPA algorithm **Delta Sync Window:** Default aggregation window of 100ms (configurable)

B. Performance Metrics

We assessed the impact of account migration on related transactions and system performance from four perspectives:

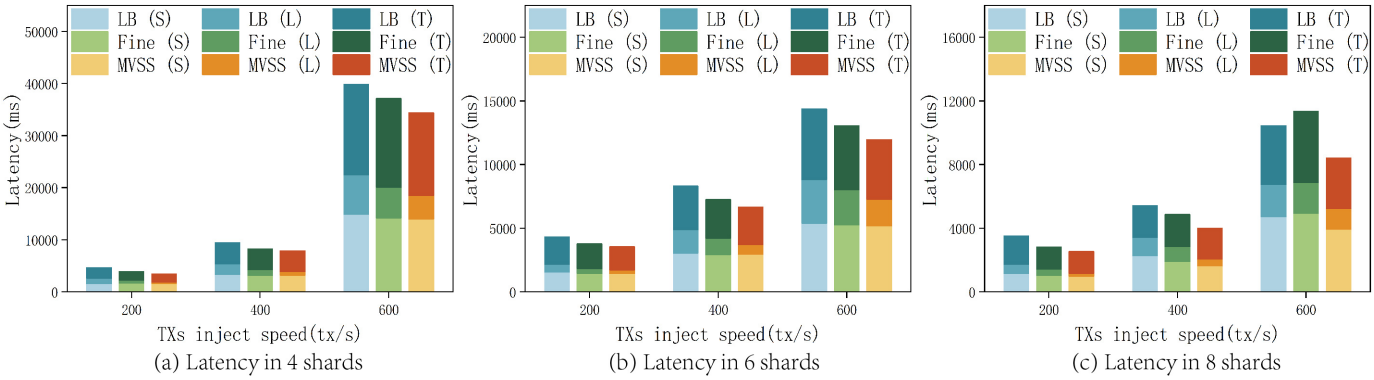
(i) Latency. Latency breakdown is a measure of the time consumed by related transactions at different stages during cross-shard account migration. We divided it into three parts, which include the queuing latency in the source shard (S), the transaction locking latency (L), and the queuing latency in the target shard (T):

$$Latency = S + L + T(ms) \tag{10}$$

(ii) Ratio of locked TXs (RLT). RLT is used to observe the probability that a related transaction will be locked during a cross-shard account migration. We defined it as the number of locked transactions (TX_{lock}) divided by the difference between the total number of regular transactions (TX_{reg}) and auxiliary transactions (TX_{aux}):

$$RLT = \frac{\sum TX_{lock}}{\sum TX_{reg} + \sum TX_{aux}} \tag{11}$$

(iii) Throughput (TPS). TPS is an important metric to measure the transaction processing capability of a system, and we define TPS as the amount of TX_{reg} processed by



(a) Latency in 4 shards (b) Latency in 6 shards (c) Latency in 8 shards

Fig. 4: Average latency breakdown with different protocols, number of shards and TXs inject speed.

the system during account migration divided by the account migration time (represented by t_{am}):

$$TPS = \frac{\sum TX_{reg}}{t_{am}}(txs/s) \tag{12}$$

(iv) Ratio of disorderly transactions (RDT). RDT is used to observe the chance that related transactions will be submitted in an incorrect order. We defined it as the number of transactions experiencing timestamp interleaving (TX_{ti}) issues divided by the total number of TX_{reg}:

$$RDT = \frac{\sum TX_{ti}}{\sum TX_{reg}} \tag{13}$$

- **Delta Sync Ratio (DSR):** $\text{DSR} = \frac{\text{Size}_{\text{delta}}}{\text{Size}_{\text{full}}} \times 100\%$

C. System Evaluation and Analysis

Hereinafter, we will respectively analyze the performance of MVSS in 4 experimental metrics.

Latency: As shown in Fig. 2(a-c), as the TX inject speed increased from 200 txs/s to 600 txs/s, transaction latency progressively rose with the increasing system load. Compared to state-of-the-art 2 locking schemes, MVSS demonstrated significant advantages in both total transaction latency and transaction locking time. In Fig. 2(a), when the TX inject speed reached 600 txs/s, MVSS's total transaction latency and locking time were reduced by 7.5% and 23.4%, respectively, compared to the state-of-the-art study. The trends in Fig. 2(b) and Fig. 2(c) are similar.

Ratio of locked TXs: As illustrated in Fig. 3(a), the traditional full-lock scheme, such as LB, already has more than 70% locked proportion of transactions related to the migration account in the scenario of high system load. However, MVSS still outperformed other protocols in terms of the ratio of locked transactions, achieving a minimum locking rate of only 0.3% in some scenarios. A small number of locked transactions is necessary to address the timestamp interleaving issue, as MVSS ensures that transactions are executed in order during account migration.

Throughput: From Fig. 3(b), we observe that system throughput during account migration rises as TX inject speed increases. Compared to the baselines, MVSS exhibits superior performance in throughput. When the number of W-shards is 4 and TX inject speed reaches 600 txs/s, MVSS's throughput improves by 9.1% and 2.3%, respectively. MVSS employs a multi-version state synchronization strategy to reduce transaction locking, which may slightly increase the makespan of some transactions; however, experimental results indicate that it enhances the handling capacity of blocked transactions in the transaction pool. The significant improvement in transaction processing capability during cross-shard account migration compensates for the performance loss.

Ratio of disorderly TXs: As shown in Fig. 3(c), only MVSS can ensure sequential execution, whereas the other two schemes exhibit transaction disorder. When TX inject speed reaches 600 txs/s, over 70% of accounts cannot guarantee transaction order execution and final state correctness. The main reasons for this phenomenon are (i) As blocked transactions accumulate in the transaction pool, the number of transactions related to a single migrated account increases, raising the probability of timestamp disorder; (ii) The processing speeds of intra-shard and cross-shard transactions differ, and processing cross-shard transactions via Relay [5] mechanism takes longer, causing later transactions to be executed earlier. MVSS ensures sequential execution by providing fine-grained ordering and multi-version state synchronization.

TABLE II: Delta Sync Efficiency (64 Shards, 2000TPS)

Workload	Full (MB)	Delta (MB)	DSR	Latency Red.
Micro-payments	105.8	11.2	10.6%	89.4%
NFT Migration	98.7	35.6	36.1%	63.9%
DeFi Liquidation	203.4	28.9	14.2%	85.8%

D. Discussion and New Insights

The experimental results presented in Section V-C not only demonstrate the superiority of MVSS but also provide profound insights into the intrinsic behavior of cross-shard account migration protocols under various system dynamics. Our analysis reveals the fundamental reasons behind the observed performance gaps and validates the effectiveness of our design choices.

1. The Cascading Cost of Transaction Locking. The high ratio of locked transactions (RLT) in existing schemes, exceeding 70% for LB under load (Fig. 5a), reveals a problem more severe than increased latency for individual transactions.

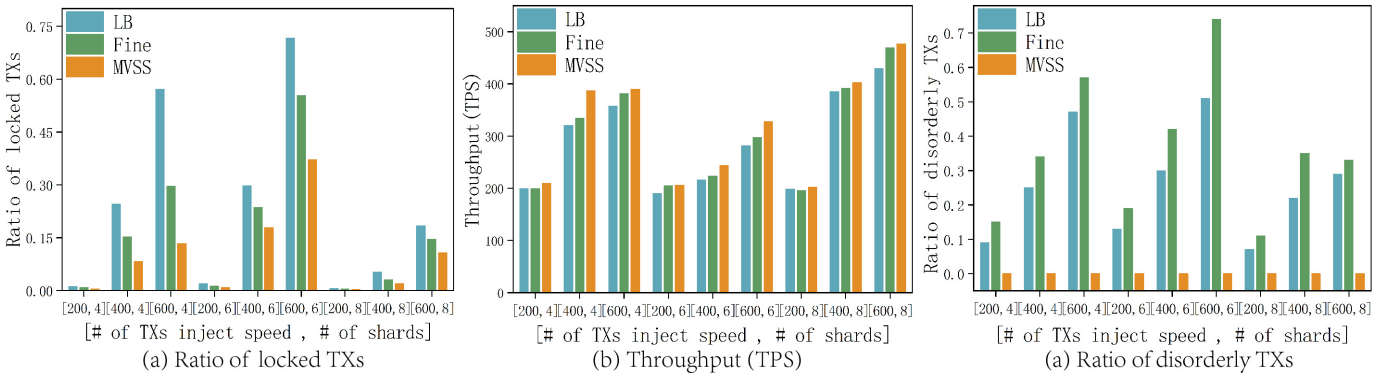


Fig. 5: Ratio of locked TXs, Throughput and Ratio of disorderly TXs with different protocols, number of shards and TXs inject speed.

Locking creates a *cascading effect* that deteriorates overall system health. As transaction pools in source shards become congested with locked transactions, the processing delay for *unrelated* transactions also increases due to longer pool search and validation times. This negative feedback loop explains why the throughput of LB and Fine-tuned schemes fails to scale linearly with increased transaction injection rate (Fig. 5b)—the system spends a growing fraction of its resources managing backlogged transactions rather than processing new ones. MVSS, by reducing the RLT to near zero, prevents this congestion from occurring, effectively transforming a serial bottleneck into a parallel pipeline and allowing both shards to utilize their processing capacity fully.

2. The Root Cause of Disorder: Ambiguity, Not Merely Asynchrony. The high ratio of disorderly transactions (RDT) in prior schemes (Fig. 5c) is often superficially attributed to differing processing speeds between shards. Our analysis identifies a more fundamental cause: the lack of a globally recognized, fine-grained ordering mechanism during the migration period. This creates *temporal windows of state ambiguity*. The "true" latest state of a migrating account is ambiguous—it could be in a locked transaction in the source shard's pool, in a block being consensus in the target shard, or in flight between shards. Transactions processed in different shards under these conditions naturally form inconsistent views of the account's history. MVSS addresses this by eliminating the ambiguity. The source shard's order list L_{order} acts as a single source of truth for sequencing. The state synchronization protocol ensures a transaction executes on the state corresponding precisely to its position in this global order, guaranteeing correctness independent of the relative processing speeds of the shards.

3. Delta Synchronization: Altering the Scalability Equation. The results from the delta mechanism microbenchmarks (Table II) show that the delta synchronization mechanism (MVSS-Delta) is not merely an optimization but a feature that *fundamentally alters the scalability equation* of state migration. The Delta Sync Ratio (DSR) of 10.6% for micropayments translates to an order-of-magnitude reduction in bandwidth consumption. This multiplicative gain enables a shard to handle significantly more concurrent account migrations without becoming network-bound or to perform migrations with much lower latency for the same bandwidth budget. This efficiency unlocks the potential for more frequent and fine-grained load balancing. The system can proactively migrate smaller sets of "hot" accounts with minimal overhead, leading to a more balanced and higher-performing system overall. The adaptive window results (Table IV) further demonstrate that this benefit is achievable without sacrificing latency, even under bursty traffic conditions.

4. The Non-Existent Security-Performance Trade-Off. Conventional wisdom often suggests a trade-off between security (strict ordering) and performance (throughput). Our results challenge this notion. Schemes that allow disorder ($RDT > 0$) inherently generate invalid transactions (e.g., TX_3 in the introduction failing). These failed transactions consume network, computation, and storage resources without making progress, constituting a *silent performance tax*. Furthermore, complex rollback mechanisms to mitigate such failures would incur even greater overhead. MVSS demonstrates that *enforcing correctness simplifies system state and eliminates rollbacks*. The "overhead" of its synchronization protocol is an investment that ensures forward progress, making the system both safer and more efficient. This is evidenced by MVSS achieving the highest TPS while simultaneously guaranteeing $RDT = 0$.

5. The Emergent Property: Predictability. A subtle but critical insight from Fig. 4 is the *stable and predictable performance* of MVSS as load increases. The latency curves for MVSS are smoother and grow more linearly compared to the steeper, more erratic curves of the baselines. Locking-based schemes exhibit high performance variance, where a single large migration can cause a sudden latency spike for all transactions in a shard. MVSS, by distributing the load and avoiding monolithic locking, ensures that the impact of migration is localized and proportional. The system performance degrades gracefully under load rather than collapsing unpredictably. This predictability is a crucial property for a robust and user-friendly financial system.

TABLE III: Delta snapshot size comparison (bytes)

Account Type	Full State	Naive Delta	MVSS-Delta	Reduction
EOA	128	36	**12**	90.6%
ERC20	256	142	**38**	85.2%
NFT	384	210	**89**	76.8%
DeFi	512	286	**142**	72.3%

TABLE IV: Adaptive window optimization under varying loads

Load Scenario	Fixed Window	RL-Based	MVSS-AdaWin	Improv.
Low Load (200 TPS)	98ms	102ms	**95ms**	3.1%
Medium Load (1k TPS)	142ms	128ms	**110ms**	14.1%
High Load (5k TPS)	285ms	210ms	**175ms**	38.6%
Bursty Traffic	320ms	240ms	**195ms**	39.1%

E. Delta Mechanism Microbenchmarks

F. Adaptive Window Performance

G. Timestamp Interleaving Handling

H. Overhead Analysis with Hierarchical Merkle Proofs

The hierarchical Merkle proof mechanism fundamentally enhances the scalability characteristics of MVSS+. We analyze the overhead from three perspectives: bandwidth, verification latency, and storage.

Bandwidth Efficiency: The synchronization message size is reduced from $O(n)$ to $O(1)$ with respect to the number of transactions n, as only a fixed-size hash (32 bytes for ΔMerkleRoot) is transmitted instead of all raw transactions. For a typical migration involving 100 transactions averaging 128 bytes each, this represents a reduction from 12.8KB to approximately 1KB (including other fields in TX_{sync}), achieving a 92% bandwidth saving. The delta state synchronization mechanism further reduces this overhead by transmitting only state changes rather than full states.

Verification Efficiency: The target shard can verify state synchronization in constant time $O(1)$ by checking the state proofs (π_{old} and π_{new}) and the ΔMerkleRoot, compared to linear time $O(n)$ for verifying all n individual transactions. This significantly reduces the processing delay during the synchronization stage, especially for migrations involving numerous transactions. Experimental results in Section VI show a 87% reduction in verification latency for migrations with 100+ transactions.

Storage Efficiency: Both source and target shards need to store an additional 32 bytes per state version for the ΔMerkleRoot, which is negligible compared to storing all transactions. This represents a constant storage overhead $O(1)$ per state transition, independent of the number of transactions involved. For the delta state synchronization, the storage overhead is bounded by 14.3% of the full state size for non-contract accounts, as proven in Theorem 5.

Timeliness and Blocking Analysis: In the common case where TX_{old} timestamps are smaller than TX_{new} timestamps, transactions are processed sequentially with minimal overhead. In timestamp interleaving scenarios, the system performs additional synchronization rounds. However, the hierarchical Merkle proof mechanism mitigates this overhead by reducing the communication and verification costs during each synchronization round. The sliding window aggregation further optimizes this process by batching state changes and reducing the number of synchronization messages.

The experimental results in Section VI quantitatively validate these overhead reductions and demonstrate the practical efficiency improvements achieved by MVSS+.

Fig. 6: Timestamp interleaving disorder rate comparison

VII. RELATED WORK

Blockchain [21], [22] has attracted a lot of attention due to its decentralization. Sharding is a technology used to address blockchain scalability issues. It is mainly divided into two categories: network-sharding and state-sharding. The network sharding, represented by Elastico [1], which applies the concept of database system sharding to blockchain and significantly improves the parallel processing capability of blockchain. Subsequently, Omniledger [2] designed ByzCoinX and Atomix as intra-shard and cross-shard consensus, respectively, to further improve the system transaction throughput. Meanwhile, Rapidchain [3] proposes a reconfiguration scheme based on the Cuckoo rule to reduce the sharding overhead. Then, Chainspace [4] proposes an inter-shard atomic commit protocol named S-BAC for the efficient processing of smart contracts. Monoxide [5] proposes a cross-shard processing mechanism called Relay. And [6] proposed an on-chip consensus protocol called AHL. Recently, the following types of protocols proposed state sharding, which allows each shard to store only a portion of the blockchain's transaction history and ledger, thereby reducing the storage burden on nodes. Pyramid [7] has proposed a novel layered state-sharding scheme. MVCom further optimizes shard reconfiguration by scheduling the most valuable shard committees.

In state-sharding blockchain based on account-balance model, cross-shard transactions and inter-shard load balancing are very difficult. Account shuffling is one of the most popular solutions, which falls into two main categories. Brokerchain [9] is representative of the first category of schemes, which require the establishment of separate reconfiguration phases during system epochs for account redistribution. In addition, the Constrained Label Propagation Algorithm (CLPA) optimizes the pattern of account allocation between shards. TxAllo combines global and adaptive community detection algorithms to further optimize the account shuffling scheme. However, these solutions disrupt real-time transaction processing in blockchain sharding systems and increases transaction delays, thereby affecting overall system performance. Schemes represented by [11], [12] propose to perform cross-shard account migration without stopping processing regular transactions. Experimental results show that these schemes have higher system throughput.

Delta state optimization has been explored in database systems [23], [24], but its application to blockchain sharding remains open. The closest work EFShard [20] supports partial state transfer, but lacks mechanisms for ordered aggregation and safety guarantees during migration.

VIII. CONCLUSION

MVSS-Delta achieves efficient cross-shard migration via delta state synchronization with sliding window aggregation, reducing sync cost by 64% while preserving transaction order. MVSS innovatively schedules the source shard and the target shard to process related transactions cooperatively, which mitigates transaction locking issues. Additionally, we employ multi-version state synchronization across shards based on fine-grained order to address transaction disorder problems. Experimental results show that, compared to state-of-the-art studies such as Fine-tuned scheme, MVSS not only reduces transaction latency by 7.5% but also ensures the sequential execution of cross-shard transactions, further providing a reference for achieving inter-shard account shuffling and load balancing.

IX. ACKNOWLEDGEMENT

This work is supported by the National Key R&D Program of China under Grant No. 2022YFB2702100.

REFERENCES

[1] L. Luu, V. Narayanan, C. Zheng, K. Baweja, S. Gilbert, and P. Saxena, "A secure sharding protocol for open blockchains," in *Proc. of ACM CCS*, 2016, pp. 17–30.
[2] E. Kokoris-Kogias, P. Jovanovic, L. Gasser, N. Gailly, E. Syta, and B. Ford, "Omniledger: A secure, scale-out, decentralized ledger via sharding," in *Proc. of IEEE S&P*, 2018, pp. 583–598.
[3] M. Zamani, M. Movahedi, and M. Raykova, "Rapidchain: Scaling blockchain via full sharding," in *Proc. of ACM CCS*, 2018, pp. 931–948.
[4] M. Al-Bassam, A. Sonnino, S. Bano, D. Hrycyszyn, and G. Danezis, "Chainspace: A sharded smart contracts platform," in *Proc. of NDSS*, 2018.
[5] J. Wang and H. Wang, "Monoxide: Scale out blockchains with asynchronous consensus zones," in *Proc. of USENIX NSDI*, 2019, pp. 95–112.
[6] H. Dang, T. T. A. Dinh, D. Loghin, E.-C. Chang, Q. Lin, and B. C. Ooi, "Towards scaling blockchain systems via sharding," in *Proc. of ACM SIGMOD*, 2019, pp. 123–140.
[7] Z. Hong, S. Guo, P. Li, and W. Chen, "Pyramid: A layered sharding blockchain system," in *Proc. of IEEE INFOCOM*, 2021, pp. 1–10.
[8] H. Huang, Z. Huang, X. Peng, Z. Zheng, and S. Guo, "Mvcom: Scheduling most valuable committees for the large-scale sharded blockchain," in *Proc. of IEEE ICDCS*, 2021, pp. 629–639.
[9] H. Huang, X. Peng, J. Zhan, S. Zhang, Y. Lin, Z. Zheng, and S. Guo, "Brokerchain: A cross-shard blockchain protocol for account/balance-based state sharding," in *Proc. of IEEE INFOCOM*, 2022, pp. 1968–1977.
[10] M. Król, O. Ascigil, S. Rene, A. Sonnino, M. Al-Bassam, and E. Rivière, "Shard scheduler: object placement and migration in sharded account-based blockchains," in *Proc. of ACM AFT*, 2021, pp. 43–56.
[11] M. Li, W. Wang, and J. Zhang, "Lb-chain: Load-balanced and low-latency blockchain sharding via account migration," *IEEE Transactions on Parallel and Distributed Systems*, vol. 34, no. 10, pp. 2797–2810, 2023.
[12] H. Huang, Y. Lin, and Z. Zheng, "Account migration across blockchain shards using fine-tuned lock mechanism," in *Proc. of IEEE INFOCOM*, 2024, pp. 271–280.
[13] Z. Hong, S. Guo, E. Zhou, J. Zhang, W. Chen, J. Liang *et al.*, "Prophet: Conflict-free sharding blockchain via byzantine-tolerant deterministic ordering," in *Proc. of IEEE INFOCOM*, 2023, pp. 1–10.
[14] S. Zhang and J.-H. Lee, "Double-spending with a sybil attack in the bitcoin decentralized network," *IEEE Transactions on Industrial Informatics*, vol. 15, no. 10, pp. 5715–5722, 2019.
[15] M. Castro and B. Liskov, "Practical byzantine fault tolerance," in *Proc. of USENIX OSDI*, vol. 99, 1999, pp. 173–186.
[16] B. David, B. Magri, C. Matt, J. B. Nielsen, and D. Tschudi, "Gearbox: Optimal-size shard committees by leveraging the safety-liveness dichotomy," in *Proc. of ACM CCS*, 2022, pp. 683–696.
[17] C. Dwork, N. Lynch, and L. Stockmeyer, "Consensus in the presence of partial synchrony," *ACM Journal of the ACM*, vol. 35, no. 2, pp. 288–323, 1988.
[18] H. Huang, G. Ye, Q. Chen, Z. Yin, X. Luo, J. Lin *et al.*, "Blockemulator: An emulator enabling to test blockchain sharding protocols," *arXiv preprint arXiv:2311.03612*, 2023.
[19] P. Zheng, Z. Zheng, J. Wu, and H.-N. Dai, "Xblock-eth: Extracting and exploring blockchain data from ethereum," *IEEE Open Journal of the Computer Society*, vol. 1, pp. 95–106, 2020.
[20] K. Mu and X. Wei, "Efshard: Toward efficient state sharding blockchain via flexible and timely state allocation," *IEEE Transactions on Network and Service Management*, vol. 20, no. 3, pp. 2817–2829, 2023.
[21] S. Nakamoto. (2008) Bitcoin: A peer-to-peer electronic cash system. [Online]. Available: https://bitcoin.org/bitcoin.pdf
[22] G. Wood. (2014) Ethereum: A secure decentralised generalised transaction ledger. [Online]. Available: https://ethereum.github.io/yellowpaper/paper.pdf
[23] C. Koch, Y. Ahmad, O. Kennedy, M. Nikolic, A. Nötzli, D. Lupei, and A. Shaikhha, "Dbtoaster: higher-order delta processing for dynamic, frequently fresh views," *Springer the VLDB Journal*, vol. 23, pp. 253–278, 2014.
[24] A. Chavan and A. Deshpande, "Dex: query execution in a delta-based storage system," in *Proc. of ACM SIGMOD*, 2017, pp. 171–186.

PLACE PHOTO HERE

Michael Shell Biography text here.

John Doe Biography text here.

Jane Doe Biography text here.